

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- **Week 3 – Logistic Regression** 
 - **Classification, sigmoid, cross-entropy loss**

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Refining Backpropagation and Deep Neural Networks (DNNs)
 - Backpropagation, chain rule, gradient descent optimization, intro to multiple layers
- Week 6 – Preventing Overfitting
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - **Assessment-1**
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - **Assessment-2**
- Week 12 - Project Presentations & Discussions (**Project Due**)

Week-3 / Agenda

Theme: Logistic REGRESSION

- Theory
 - Recap - Week 2 on Linear Regression
 - Logistic Regression for Classification
 - Logistic Regression Model
 - Log loss as Cost Function
 - Gradient Descent with Log Loss
 - Nonlinear Logistic Regression
 - Multinomial Logistic Regression
- Lab Work





Main Components of a Learning Algorithm

A learning algorithm consists of:

- A model (with learnable parameters)
 - Function that maps inputs to outputs
 - Contains parameters - values - that we don't know but must learn from the data
- A loss / cost function (measures error)
 - Measures how wrong models predictions compared to ground truth
 - Tells us how poorly a model is performing
- An optimization method (updates parameters)
 - Procedure that tells how to change parameters to reduce the cost
 - Repeatedly update parameters to reduce the cost



Linear Regression

- LR models the relationship between a target and features using a parameterized function
 - function is linear in parameters
- Model: $Y = wX + b$ Y is the target variable, X is the input variable
 - Parameters to find: w, b

- Cost Function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(h_{\theta}(x^{(i)}), y^{(i)})$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=0}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



Linear Regression (LR)

- Optimization algorithm
 - guide us downhill toward the minimum of the cost function.
- most commonly used optimization algorithm is **Gradient Descent**.
 - Gradient Descent checks how cost changes using the **derivatives** of the model function wrt parameters
 - Derivatives tell us 1. Rate of change 2. Direction of change



Linear Regression (LR)

Derivatives

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Parameter Update

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

The most common ways to **stop gradient descent** are:

1. Gradient Magnitude:

- Stop when the **overall gradient vector is very small**:
 $\|\nabla_{\theta} J\| < \epsilon$
- Means the slope is nearly flat in all directions.

2. Cost Function Change:

- Stop when the **change in cost** between steps is tiny:
 $|J(\theta_{t+1}) - J(\theta_t)| < \epsilon$
- Further updates won't meaningfully reduce the cost.

3. Maximum Iterations / Epochs:

- Stop after a predefined number of steps to prevent infinite loops.

4. Combination / Convergence Criteria:

- Often use both gradient magnitude **and** cost change together for robust stopping.



We also talked about...

- Normalization
 - Min max
 - Z score
- Multivariate Regression
- Polynomial Regression

Min max normalization

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Z-score normalization

$$x' = \frac{x - \mu}{\sigma}$$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots + \theta_n x^n$$

Lets revisit classification

- Classification is a machine learning task where the goal is to assign data points to predefined categories (classes).
- In simple terms: *You ask the model to decide which group something belongs to.*
- Examples:
 - Email → spam or not spam
 - Image → cat or dog
 - Transaction → fraud or not fraud
 - Patient → disease or no disease

So the key difference:

- Classification → target is a category
- Regression → target is a number

Whereas **regression** is a machine learning task where the goal is to predict a continuous numerical value. You ask the model to predict a number.

Examples: House price → \$245,000, Temperature tomorrow → 18.7°C, Sales next month → 1,320 units

From Linear Regression to Logistic Regression

- Linear Regression is used for regression, not classification.
- Understanding its limitations in solving classification problems naturally motivates the use of Logistic Regression for classification tasks.
- What Linear Regression Assumes
 - Linearity – The relationship between the independent variables and the dependent variable is linear.

Inputs: $X \in \mathbb{R}^n$

Output: $Y \in \mathbb{R}$

model

$$h_{\theta}(X) = \theta^T X$$

Linear Regression assumes a straight-line relationship and outputs continuous values, which makes it unsuitable for classification where discrete classes or probabilities are needed.

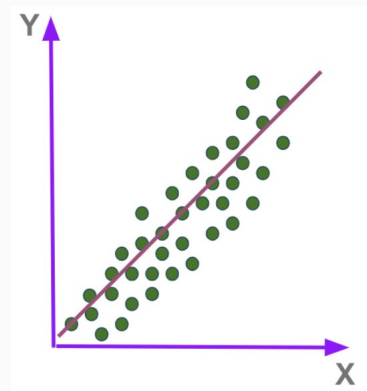
What Linear Regression Actually Doing

- Linear Regression predicts the average value of Y given X
- It places a line through the middle of the data
- This corresponds to estimating:

Linear regression tries to fit a line that best models the relationship between the input features and a target variable, **without using or considering classes.**

$$\mathbb{E}[Y \mid X]$$

**expected value/mean of Y
given X**



Classification is a Different Problem

Classification asks a fundamentally different question:

Which class does this example belong to?

- The answer must be discrete
 - a. For binary classification, the output is 0 or 1, not a middle value
- Linear Regression tries to predict the mean of the data
- The mean tells us where the center of the data lies
- Classification requires learning a boundary between classes, not the center

Example

Problem Definition: understanding the relationship between average daily consumption of added sugar and the onset of Type II diabetes.

Each individual in our dataset is described by:

- Input feature (X): Average daily sugar consumption (in grams)
- Target variable (Y): Diagnosis of Type II diabetes
 - 0 → Not diagnosed
 - 1 → Diagnosed

Our goal is to predict whether an individual has Type II diabetes based on their average daily sugar consumption.

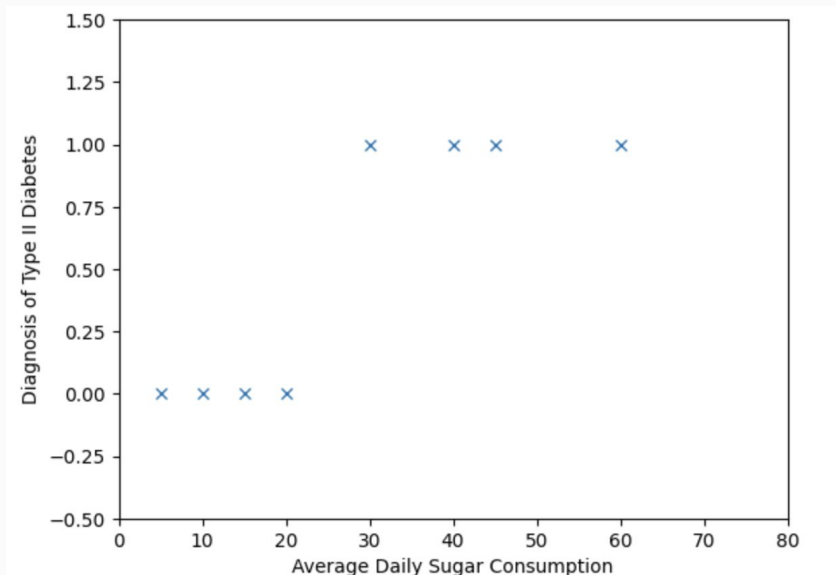
Let's check the data

Target Variable - diagnosed onset of Type II diabetes

`Y = np.array([0, 0, 0, 0, 1, 1, 1, 1])`

Input Feature - Average daily consumption of added sugars in grammes

`X = np.array([5, 10, 15, 20, 30, 40, 45, 60])`



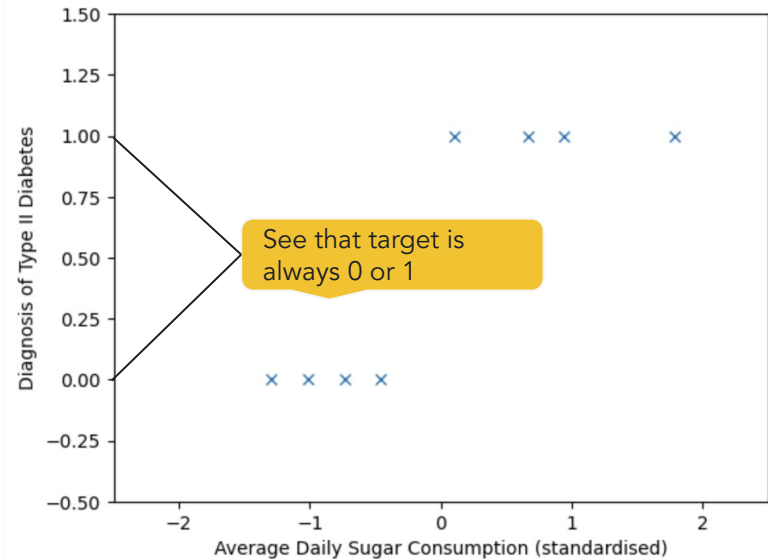
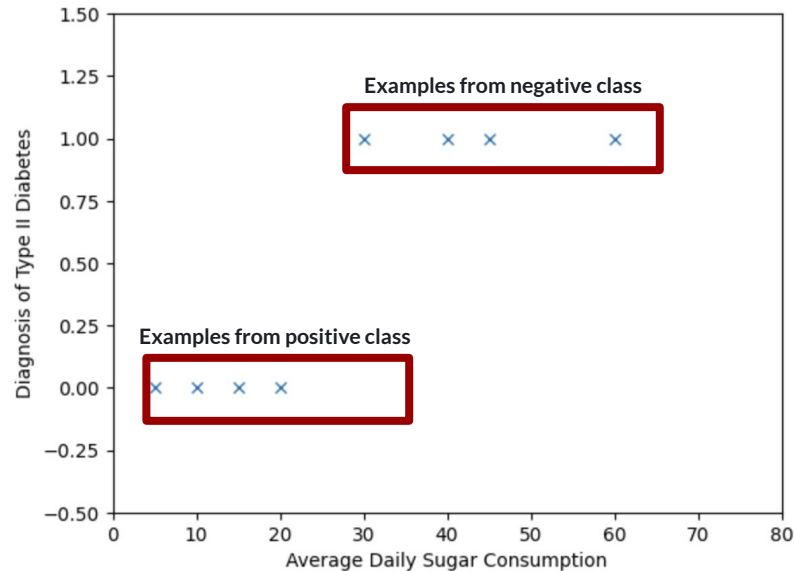
Let's check the data

Target Variable - diagnosed onset of Type II diabetes

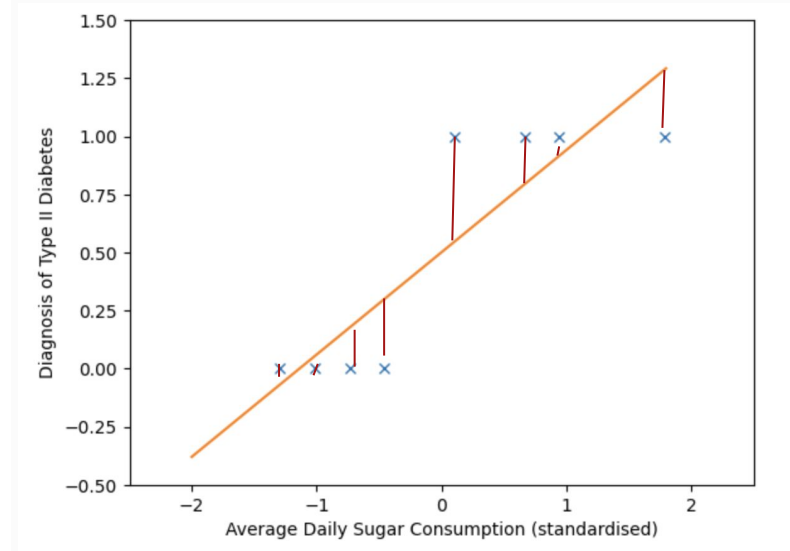
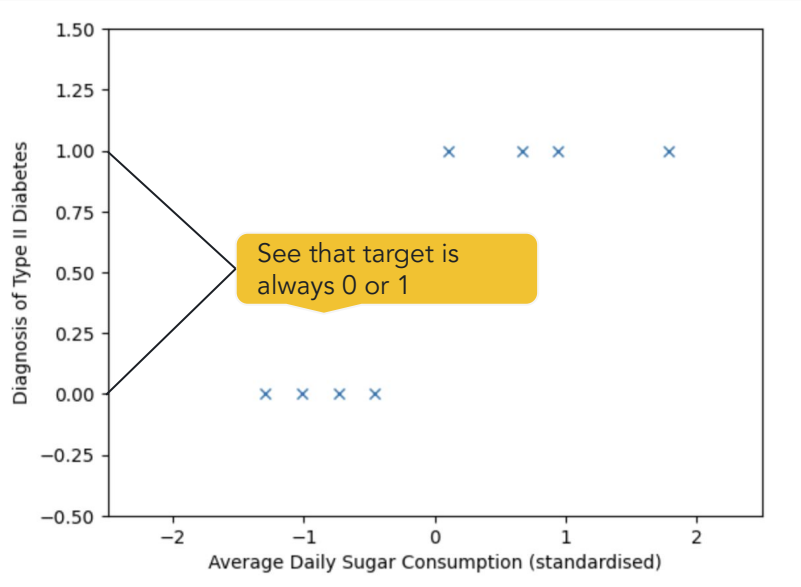
```
Y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

Input Feature - Average daily consumption of added sugars in grammes

```
X = np.array([5, 10, 15, 20, 30, 40, 45, 60])
```



If we apply linear regression

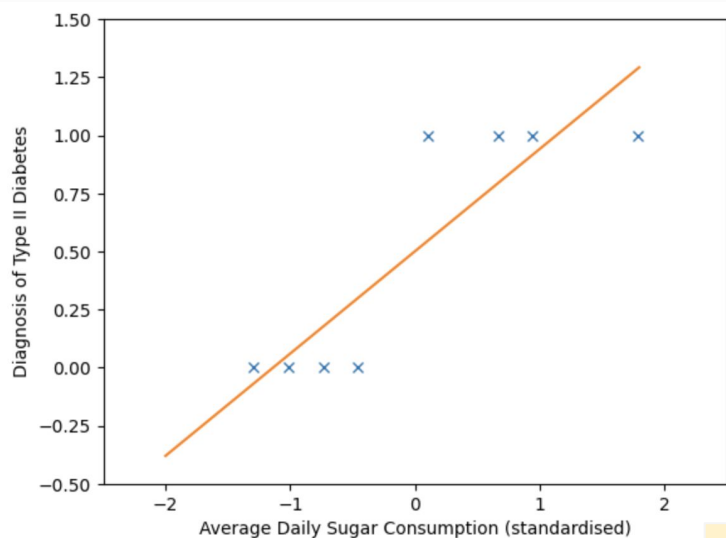


Regression Line:

- pass as close as possible to all points labeled 0
- and as close as possible to all points labeled 1
- in an average sense

That's why it often runs **through the middle of the points**, rather than between classes.

Lets use a threshold attached to the output



The **output of a Linear Regression model is unbounded**, meaning it can take **any real value**, from $-\infty$ to $+\infty$.

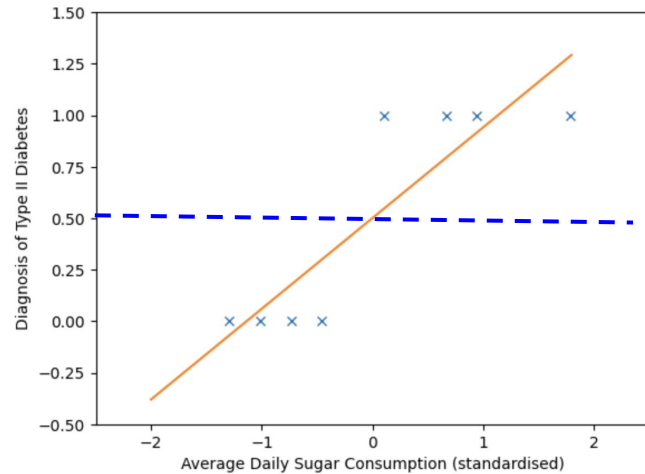
Looking at the output of our Linear Regression function, we are reminded that its range is $\mathbb{R}$.

- not well suited to a classification task, since the model can produce values below 0 or above 1.
- we can address this limitation by adopting a simple thresholding policy such that if:

$$h_{\theta}(x) \geq 0.5 \text{ then } y = 1$$

$$h_{\theta}(x) \leq 0.5 \text{ then } y = 0$$

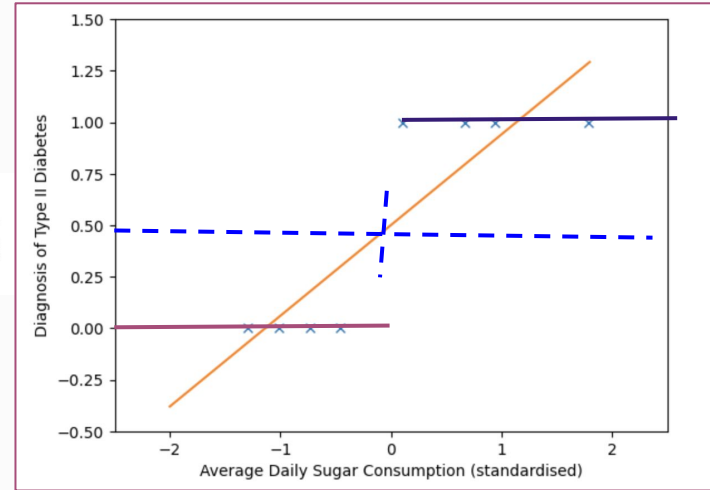
Lets use a threshold attached to the output



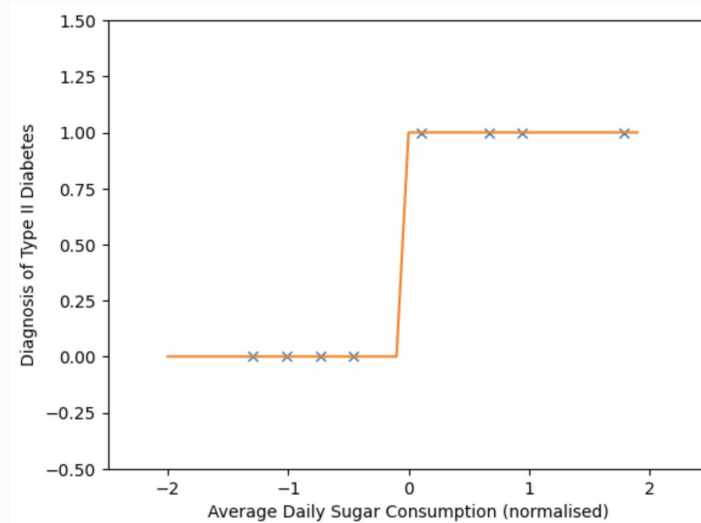
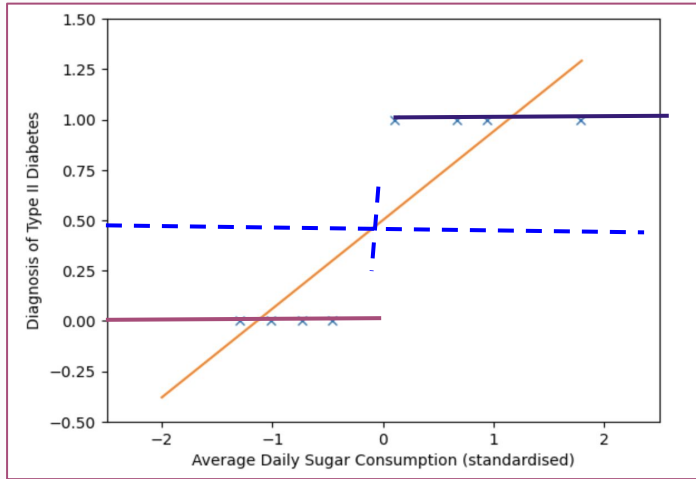
APPLY

$h_{\theta}(x) \geq 0.5$ then $y = 1$

$h_{\theta}(x) \leq 0.5$ then $y = 0$

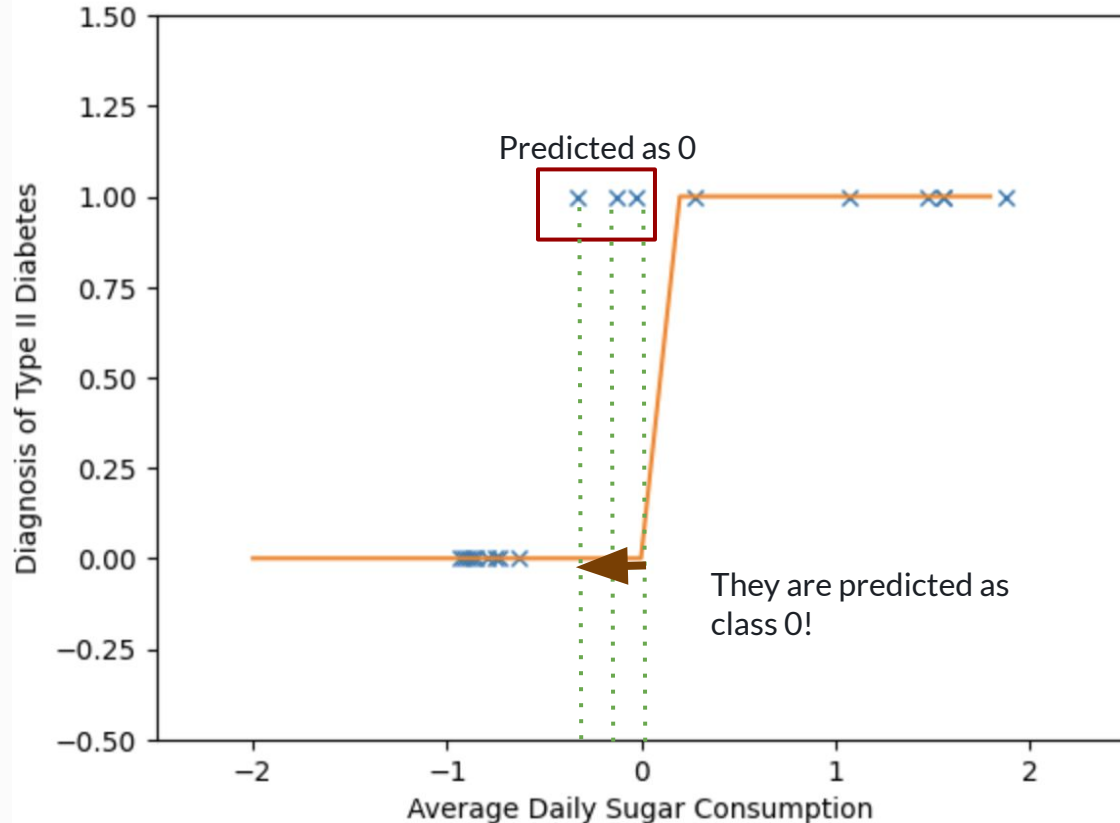


Lets use a threshold attached to the output



- At first glance, the output of our linear regression model appears to fit the data reasonably well
- If the classes overlap or the data is noisy, many points from different classes lie in the same region of the input space. The model therefore places the line through this mixed region

How Linear Regression Fails for Classification



- Even though data has a clear separation, linear regression cannot see the distinction between classes.
- The fitted line passes through regions containing both classes, and thresholding cannot fix this because the model was never trained to find a decision boundary.

Let's Solve this Using a Classification Algorithm

- Logistic Regression -

- A classification algorithm specifically designed for binary outcomes.
- Instead of predicting numeric values, it predicts the probability of belonging to class 1.
- Uses the Logistic Function (Sigmoid) with cross entropy loss:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Logistic Regression (Linear Regression + Sigmoid)

Logistic Regression (Linear Regression + Sigmoid)

1. Linear combination of features

Logistic Regression first computes a linear combination of the input features:

$$z = \theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

2. Apply the sigmoid function

The hypothesis function $h_\theta(x)$ maps this linear combination to a probability:

$$h_\theta(x) = \sigma(z) = \frac{1}{1 + e^{-\theta^T x}}$$

- Now $h_\theta(x)$ is **between 0 and 1**, representing the probability that $y = 1$.

3. Training with cross-entropy loss

The model is trained using **cross-entropy loss**, which encourages it to **learn a boundary that separates the classes**, rather than just approximating the mean of y .

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

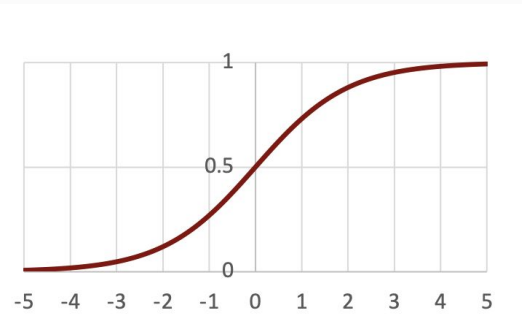
- Unlike mean squared error (which treats outputs like continuous values), cross-entropy **focuses on matching probabilities to the actual classes**, not approximating a mean.
- **cross-entropy pushes the model to place the boundary where it best separates the two classes.**

The Logistic (Sigmoid) Function

- Smooth, continuous S-shaped curve, acts like a soft threshold
 - output of the sigmoid changes gradually as the input changes. There's no sudden jump, small changes in input produce small changes in output.
- Always between 0 and 1

Why this matters:

- Perfect for modeling probabilities
- Differentiable → trainable



LOGISTIC REGRESSION MODEL

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$



$$h_{\theta}(x) = g(\theta^T x)$$

The Logistic (Sigmoid) Function

- Smooth, continuous S-shaped curve
- Always between 0 and 1
- Looks like a soft threshold

Interpretation:

- $\theta^T x \rightarrow$ linear score
- $g(\cdot) \rightarrow$ converts score to probability

Key idea:

Logistic regression = **linear model + sigmoid**

LOGISTIC REGRESSION MODEL

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$



$$h_{\theta}(x) = g(\theta^T x)$$

Why Sigmoid Is So Important

- Converts linear outputs into probabilities
- Enables meaningful loss functions (log loss)
- Used in:
 - Logistic Regression
 - Neural networks
 - Deep learning classifiers



Callout:
This function appears
everywhere in ML — good
to learn it well.

$$g(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function Parameters

Assume a **single-feature** logistic regression model:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x)$$

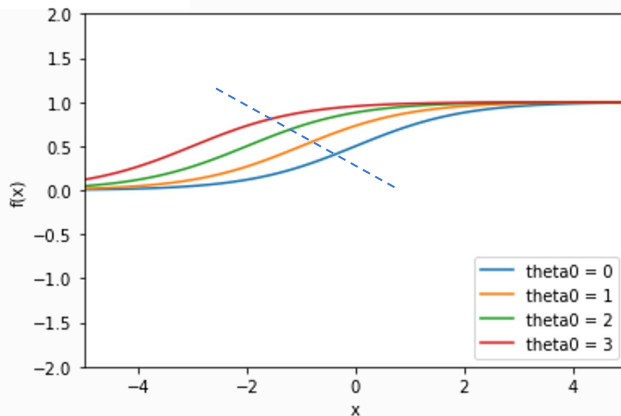
Here:

- θ_0 = **intercept** (horizontal shift)
- θ_1 = **slope / weight of the feature**

θ_0 : shifts the curve left or right

- Changes **where** the transition happens
- Does **not** change how steep the transition is

Think: *where is the 0.5 point?*



SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$

LOGISTIC REGRESSION

$$h_{\theta}(x) = g(\theta^T x)$$

Sigmoid Function Parameters

Assume a **single-feature** logistic regression model:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x)$$

Here:

- θ_0 = **intercept** (horizontal shift)
- θ_1 = **slope / weight of the feature**

θ_1 : **controls the steepness**

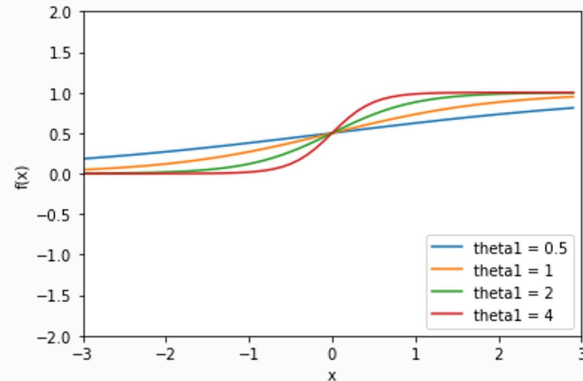
- Scales how fast $z = \theta_0 + \theta_1 x$ grows
- Directly affects how quickly sigmoid moves from 0 $\rightarrow$ 1

SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$

LOGISTIC REGRESSION

$$h_{\theta}(x) = g(\theta^T x)$$



Sigmoid Functions

- logistic function is one specific example of a broader family of functions known as **sigmoid functions**.
- sigmoid functions are characterised by
 - their S-shaped curves and
 - by the fact that they smoothly transition between two limiting

- As the input $z \rightarrow -\infty$, the output approaches the **lower limit** (e.g., 0)
- As the input $z \rightarrow +\infty$, the output approaches the **upper limit** (e.g., 1)
- The curve doesn't jump abruptly—it **gradually moves** from the lower to upper value

SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$

LOGISTIC REGRESSION

$$h_{\theta}(x) = g(\theta^T x)$$

Logistic Functions and Sigmoid Functions

- logistic function is one specific example of a broader family of functions known as **sigmoid functions**.
- sigmoid functions are characterised by
 - their S-shaped curves and
 - by the fact that they smoothly transition between two limiting

What does “smooth transition between two limiting” mean?

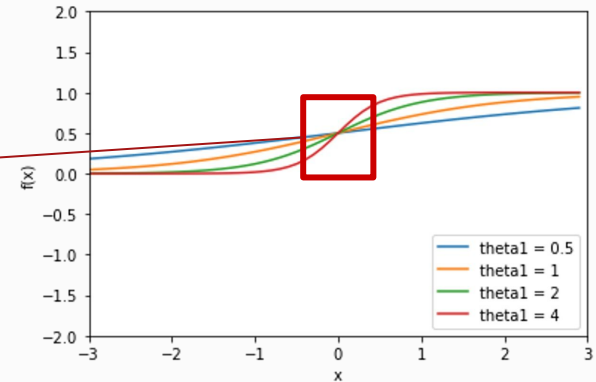
- transition is the region where the function moves from one extreme value → the other
- For sigmoid functions, this usually means: from “low” output (near 0 or -1) to “high” output (near 1)
- For sigmoid: Left side: confident class 0, Middle: uncertainty, Right side: confident class 1
- the transition region is where the boundary lies, separating class 0 from class 1.

SIGMOID FUNCTION

$$g(x) = \frac{1}{1 + e^{-x}}$$

LOGISTIC REGRESSION

$$h_{\theta}(x) = g(\theta^T x)$$



Logistic Functions and Sigmoid Functions

- logistic function is one specific example of a broader family of functions known as **sigmoid functions**.
- sigmoid functions are characterised by
 - their S-shaped curves and
 - by the fact that they smoothly transition between two limiting

SIGMOID FUNCION

$$g(x) = \frac{1}{1 + e^{-x}}$$

LOGISTIC REGRESSION

$$h_{\theta}(x) = g(\theta^T x)$$

Why is smooth transition important?

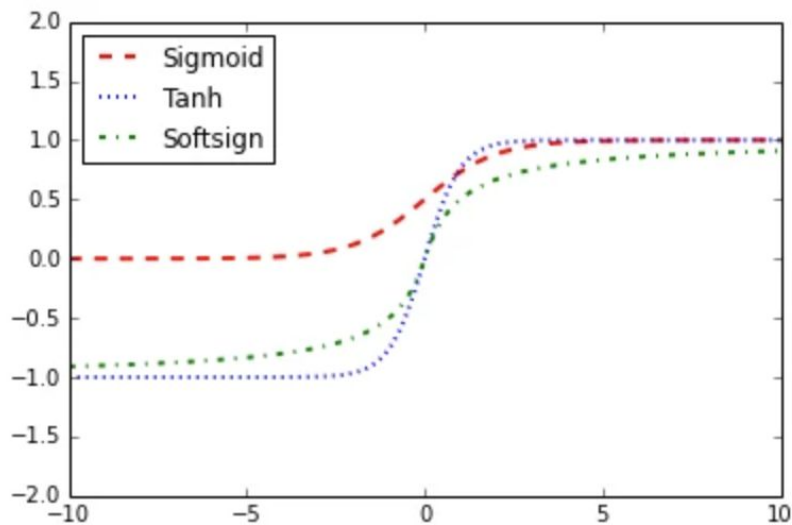
- A function is smooth if it has no sharp corners, no jumps or sudden breaks
- Meaning Its slope changes gradually
- Mathematically function is continuous and Its derivative exists everywhere



What "smoothness" matter in ML?

- Gradient descent needs derivatives
- Sharp jumps = no gradient = no learning

Other Sigmoid Functions



Sigmoid functions allow models to learn smooth transitions and probabilistic or bounded outputs.

- Logistic Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma(x) \in (0, 1)$$

- Hyperbolic Tangent (tanh)

$$\tanh(x), \quad \tanh(x) \in (-1, 1)$$

- Softsign

$$f(x) = \frac{x}{1 + |x|}, \quad f(x) \in (-1, 1)$$

Where used:

Function	Typical Use	Output Range
Logistic Sigmoid	Binary classification, Logistic Regression	(0, 1)
Tanh	Hidden layers in neural networks	(-1, 1)
Softsign	Alternative neural network activation	(-1, 1)

Logistic Regression as Probabilities & Decisions

- Logistic function outputs values in **[0,1]**, allowing a **probabilistic interpretation**
- Hypothesis function:

$$h_{\theta}(x) = P(Y = 1 \mid x, \theta)$$

- Probability of the other class:

$$P(Y = 0 \mid x, \theta) = 1 - h_{\theta}(x)$$

- Decision boundary: classify as

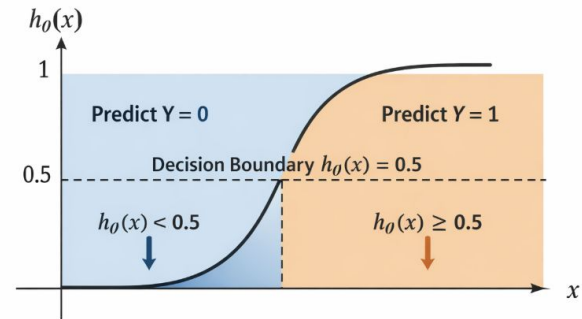
$$\hat{Y} = \begin{cases} 1 & \text{if } h_{\theta}(x) \geq 0.5 \\ 0 & \text{if } h_{\theta}(x) < 0.5 \end{cases}$$

0.5 is the default decision threshold.

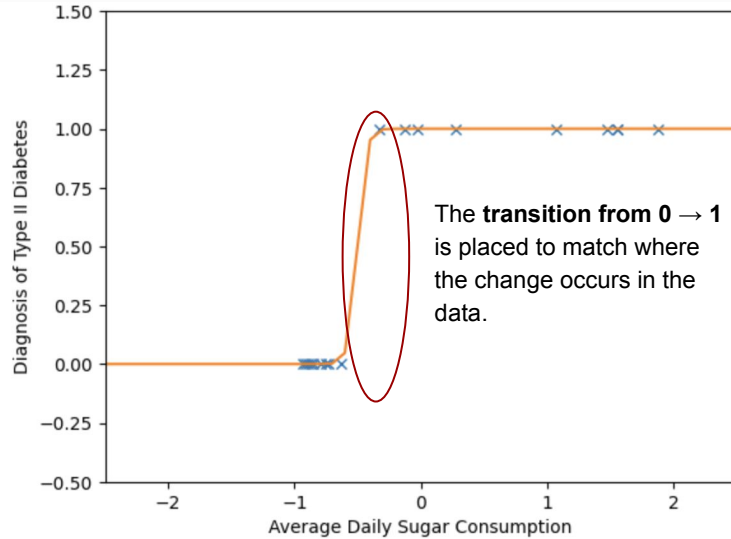
Logistic Regression is flexible, the threshold for classifying “positive” can be chosen depending on the problem, which moves the decision boundary accordingly.

$$h_{\theta}(x) = \sigma(z) = \frac{1}{1 + e^{-\theta^T x}}$$

- Logistic regression transforms a linear score into a probability
- then uses a threshold to decide the class.



Lets apply logistic function to diabetes data



VS



logistic function provides a **much better fit** than a simple thresholded linear function.

The **transition from 0 → 1** is placed to match where the change occurs in the data.

Logistic sigmoid applied to dataset

Components of Logistic Regression as a Learning Algorithm

- A model (with learnable parameters)
- A loss / cost function (measures error)
- An optimization method (updates parameters)

Components of Logistic Regression as a Learning Algorithm

- Model -

1. The Model:

- A logistic (sigmoid) function that maps the linear combination of input features to a probability:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

- θ are the learnable parameters of the model.

Components of Logistic Regression as a Learning Algorithm

- Cost Function -

- We cannot use MSE because for logistic regression model:
 - The resulting cost function is non-convex meaning the error surface may have multiple local minima, so gradient descent can get stuck in a suboptimal solution.

$$J_{\text{MSE}}(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

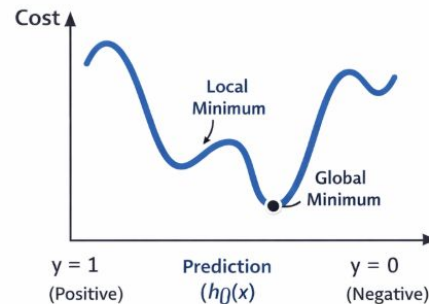


$$(h_{\theta}(x) - y)^2 = (\sigma(\theta^T x) - y)^2$$



MSE as cost with
sigmoid as the model
function is non-convex!

MSE Cost for Logistic Regression



Non-Convex: Multiple Minima

Instead we use Log Loss (aka Cross-Entropy) Loss Function

For logistic regression, we define the Loss function (cost for a single training):

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$$

- Despite its appearance, it is simple to interpret and well-suited for probabilistic output
- The cost function behaves **differently depending on the label:**
 - One term applies when $y=1$
 - The other applies when $y=0$

Instead we use Log Loss (aka Cross-Entropy) Function

For logistic regression, we define the Loss function (cost for a single training):

If $y=1$, this evaluates to 0,
cancelled out!

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \underbrace{-y^{(i)} \log(h_{\theta}(x^{(i)}))}_{\text{Applies when } y=1} - \underbrace{(1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))}_{\text{If } y=1, this evaluates to 0, cancelled out!}}$$

- Despite its appearance, it is simple to interpret and well-suited for probabilistic output
- The cost function behaves differently depending on the label:
 - One term applies when $y=1$
 - The other applies when $y=0$

Cost when true label $y=1$

For logistic regression, we define the Loss function (cost for a single training):

If $y=1$, this evaluates to 0,
cancelled out!

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \boxed{-y^{(i)} \log(h_{\theta}(x^{(i)}))} - \boxed{(1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))}$$

Applies when $y=1$

For $y = 1$, the logistic cost is:

$$\text{cost}(h_{\theta}(x), 1) = -\log(h_{\theta}(x))$$

CASE 1: If model predicts 0

If

$$h_{\theta}(x) \approx 0 \Rightarrow \log(0) \rightarrow -\infty$$

So the **cost is very high**.

- ✗ Model is confidently wrong
- ✗ Heavy penalty

If the true label is 1 and the model predicts 0, the loss is infinite — the worst possible error.

Cost when true label $y=1$

For logistic regression, we define the Loss function (cost for a single training):

If $y=1$, this evaluates to 0,
cancelled out!

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \boxed{-y^{(i)} \log(h_{\theta}(x^{(i)}))} - \boxed{(1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))}$$

Applies when $y=1$

For $y = 1$, the logistic cost is:

$$\text{cost}(h_{\theta}(x), 1) = -\log(h_{\theta}(x))$$

CASE 2: If model predicts 1

Substituting $h_{\theta}(x) = 1$:

$$-\log(1) = 0$$

If the true label is 1 and the model predicts 1, the logistic cost is 0 (minimum possible loss).

Cost when true label is $y=0$

For logistic regression, we define the Loss function (cost for a single training):

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \boxed{-y^{(i)} \log(h_{\theta}(x^{(i)}))} - \boxed{(1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))}$$

If $y=0$, this evaluates to 0, cancelled out!

Applies when $y=0$

Evaluates to 1

For $y = 0$, the **logistic (cross-entropy) cost** is:

$$\text{cost}(h_{\theta}(x), 0) = -\log(1 - h_{\theta}(x))$$

- If $h_{\theta}(x) \approx 0$:

$$-\log(1) = 0$$

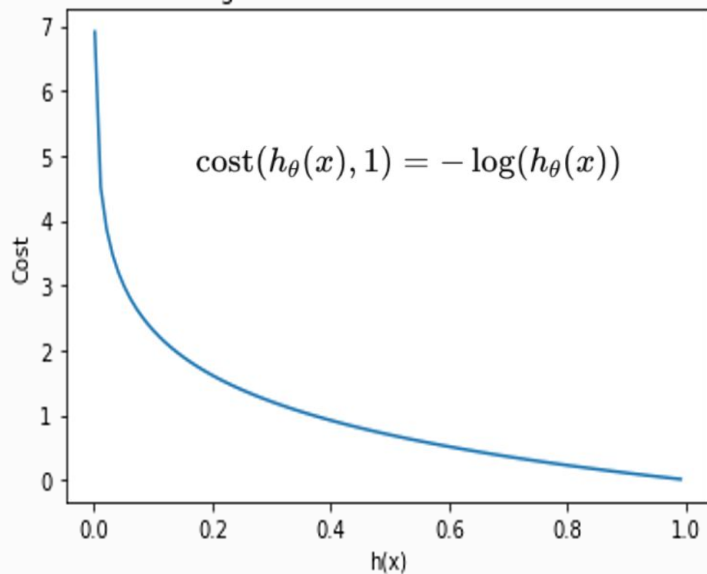
✓ Correct prediction → **low cost**

- If $h_{\theta}(x) \approx 1$:

$$-\log(0) \rightarrow \infty$$

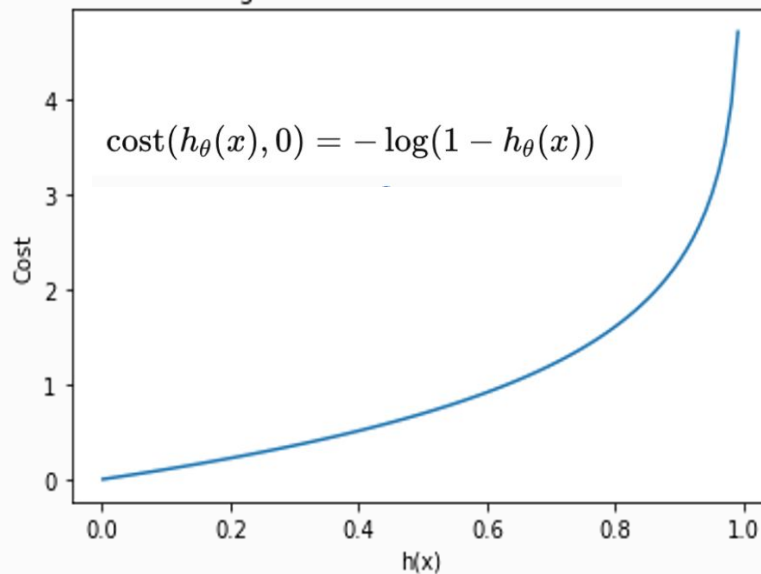
$y=0$, the cost is low if the model predicts 0 and grows rapidly as the model predicts 1.

Logistic Cost Function where Y=1



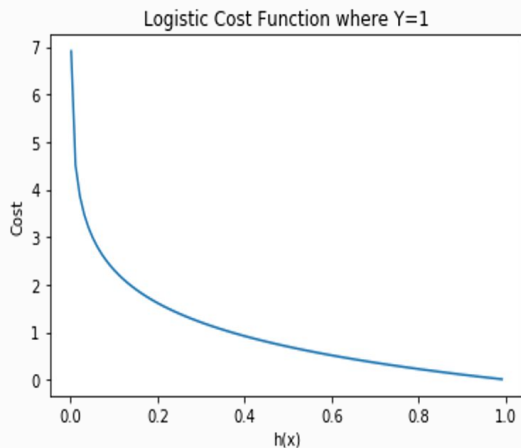
- curve is a **monotonically decreasing convex function**
- approaches infinity as predicted probability goes to 0
- **approaches zero as probability approaches 1**, heavily penalizing confident misclassifications.

Logistic Cost Function where Y=0

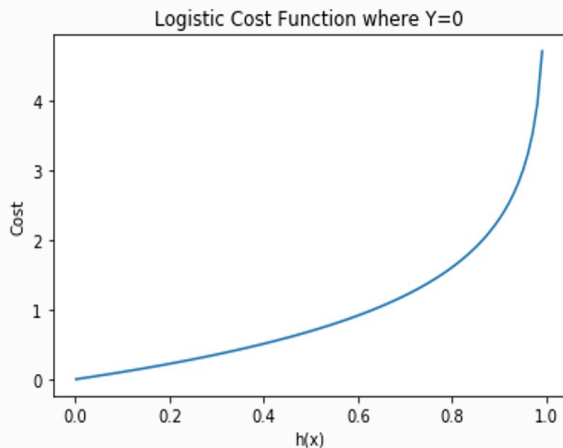


- curve is a **monotonically increasing, convex function**
- approaches zero as predicted probability goes to 0
- approaches infinity as probability approaches 1, heavily penalizing confident misclassifications.

$$\text{cost}(h_{\theta}(x), 1) = -\log(h_{\theta}(x))$$



$$\text{cost}(h_{\theta}(x), 0) = -\log(1 - h_{\theta}(x))$$



Why cost if not capped and goes to infinity?

1. It forbids confident wrong predictions

If the cost were capped (say at 10 or 100):

- The model could confidently predict 0 for a true 1
- And not be punished "enough"
- So gradient descent might **not strongly correct it**

2. It creates strong gradients where we need them most

As $h \rightarrow 0$:
when $y=1$

$$\frac{d}{dh} [-\log(h)] = -\frac{1}{h} \rightarrow -\infty$$

Meaning:

- **Huge gradient**
- Big parameter update
- Fast correction of catastrophic mistakes

1. Sigmoid never outputs exactly 0 or 1

$$0 < h_{\theta}(x) < 1$$

2. We use numerical clipping in real implementations:

python

```
h = np.clip(h, 1e-15, 1 - 1e-15)
```

Cross Entropy Cost Behavior

Let's see numerical examples:

$$\text{cost}(h_{\theta}(x^{(i)}), y^{(i)}) = -y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$$

True label y	Predicted $h_{\theta}(x)$	Cost $J(h, y)$	Interpretation
1	1	0	Perfect prediction
1	0.9	0.105	Slightly off
1	0.5	0.693	Uncertain, moderate cost
1	0.1	2.303	Confident but wrong, high cost
1	0	∞	Completely wrong, maximum penalty
0	0	0	Perfect prediction
0	0.1	0.105	Slightly off
0	0.5	0.693	Uncertain, moderate cost
0	0.9	2.303	Confident but wrong, high cost
0	1	∞	Completely wrong, maximum penalty

- Unlike mean squared error (which treats outputs like continuous values), cross-entropy **focuses on matching probabilities to the actual classes**, not approximating a mean.
- **cross-entropy pushes the model to place the boundary where it best separates the two classes.**

Log Loss Cost Function



Log loss is ideal because it rewards correct probability estimates and heavily penalizes confident mistakes, leading to stable and meaningful learning.

1. Cost = 0 when prediction is perfect

- $h = 1$ if $y = 1$, $h = 0$ if $y = 0$

2. Cost grows rapidly when confident but wrong

- $h \rightarrow 0$ when $y = 1$
- $h \rightarrow 1$ when $y = 0$

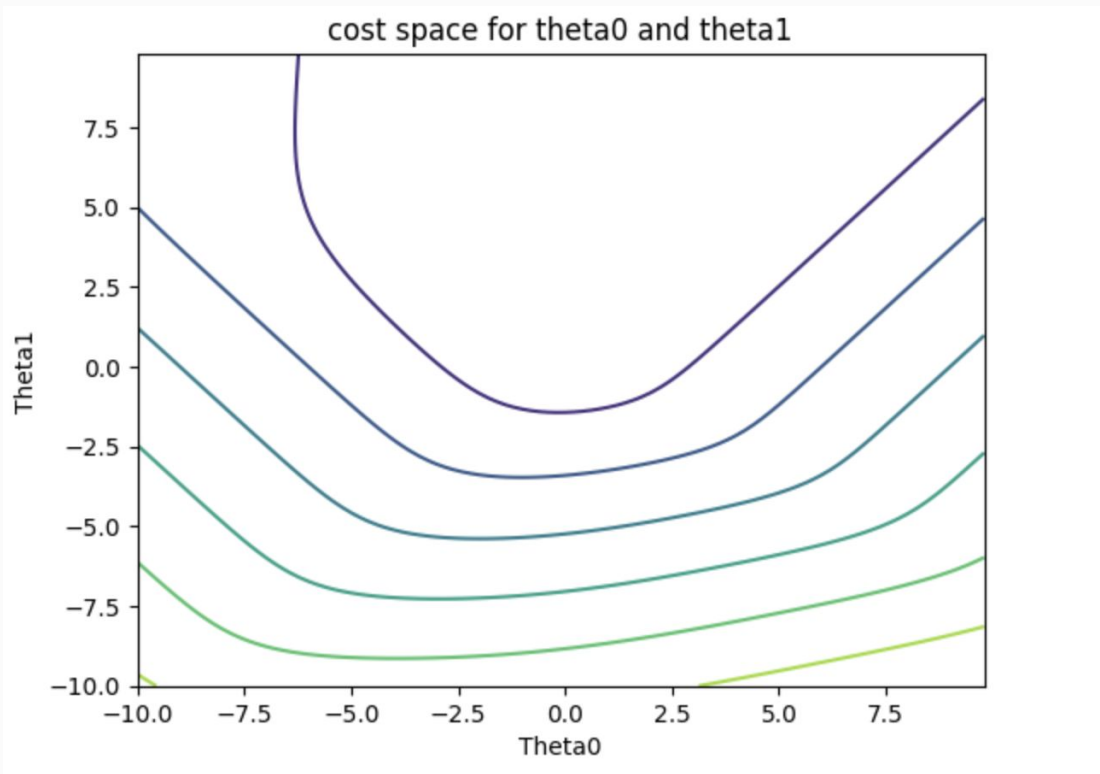
3. Moderate cost when uncertain

- $h \approx 0.5 \rightarrow \text{cost} \approx 0.693$
- Encourages model to become more confident in correct predictions

4. Convex

- Guarantees a single global minimum for gradient descent

Cost Surface



- Each point on the plot represents a specific set of parameters
- Contour lines show all parameter sets with the same cost.
- The lowest point on the surface = parameters that minimize the cost → best model fit
- The shape of the contours tells us about the sensitivity of the cost:
 - Close contours → cost changes rapidly (steep)
 - Wide contours → cost changes slowly (flat)
- Gradient descent works by following the slope of this surface down to the minimum.

Optimization: Gradient Descent

3. Optimization Method:

- An algorithm to update θ to minimize the cost function.
- **Gradient descent** (or variants like stochastic or mini-batch) is commonly used:

$$\theta := \theta - \alpha \nabla_{\theta} J(\theta)$$

where α is the learning rate.

Gradient Descent for Logistic Regression

- **Idea:** Gradient descent minimizes a cost function $J(\theta)$ by updating parameters θ in the direction of steepest descent (negative gradient).
- In **linear regression**, we used the partial derivatives of the squared error cost function.
- **In logistic regression:**
 - The same principle applies: we update θ using the **partial derivatives of the logistic regression cost function**.
 - The only difference is that the **hypothesis function** is now the sigmoid:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Gradient Descent Partial Derivatives

The cost for a single training example is:

$$\text{cost}_{\text{logistic}}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

The overall cost is the **average over all examples**:

$$J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m \text{cost}_{\text{logistic}}(h_{\theta}(x^{(i)}), y^{(i)})$$



See [Appendix](#) for Math Details, and feel free to ask any questions there!

The partial derivative of the logistic cost with respect to θ_j is:

$$\frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Gradient Descent Update Formula

The partial derivative of the logistic cost with respect to θ_j is:

$$\frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta)$$

- α is the learning rate
- This update is applied **simultaneously for all** θ_j
- Repeating this iteratively moves θ toward the **minimum of** $J_{\text{logistic}}(\theta)$

Gradient Descent for Logistic Regression

1. Initialize $\theta = 0$ or small random values

2. Repeat until convergence:

- Compute hypothesis $h_{\theta}(x^{(i)})$ for all training examples
- Compute the gradient:

$$g_j = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- Update parameters:

$$\theta_j := \theta_j - \alpha g_j$$

3. Return optimized θ

Multivariate Logistic Regression

- Extending logistic regression to multiple input features
- From 1 Feature to Many

- So far, logistic regression used **one feature** X to predict a binary outcome Y .
- Multivariate logistic regression uses **multiple features**, e.g.:

$$X_1 = \text{Average Daily Sugar Consumption}, \quad X_2 = \text{BMI}$$

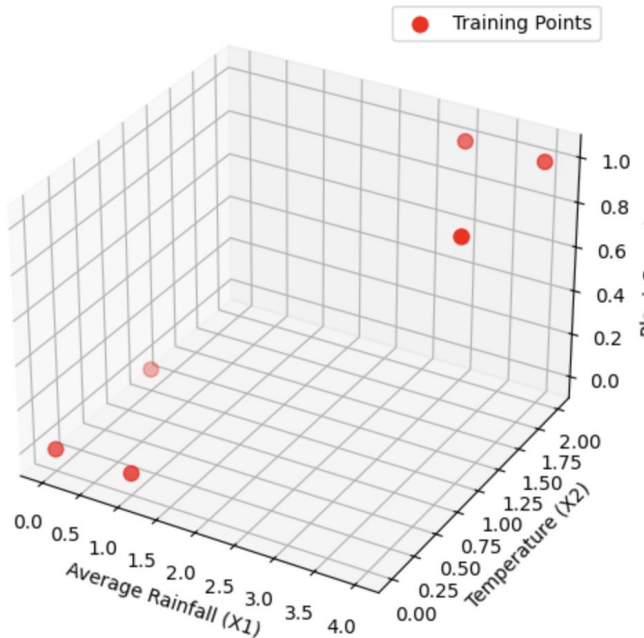
- The hypothesis becomes:

$$h_{\theta}(X) = g(\theta_0 + \theta_1 X_1 + \theta_2 X_2)$$

- Where $g(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

Multivariate Logistic Regression

Training Data: Features vs True Labels



Predicting **Plant Growth (Y)** as a binary outcome:

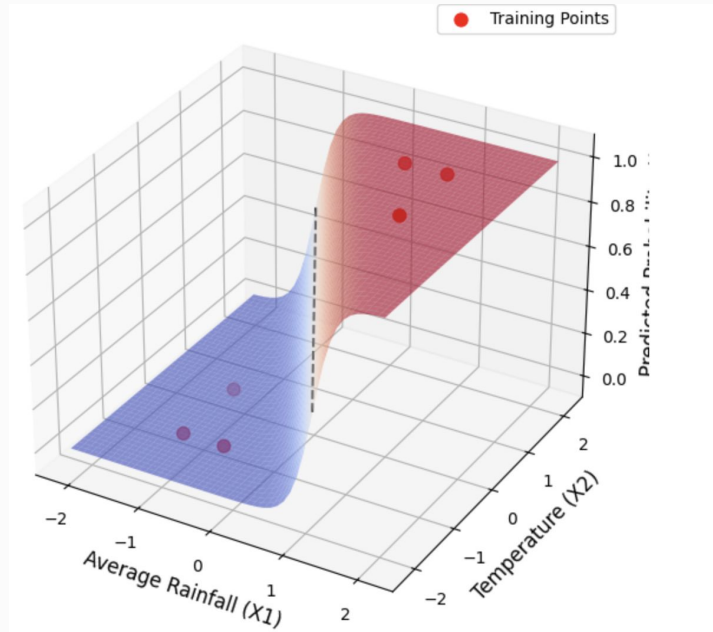
- " $Y = 1 \rightarrow$ plant grows successfully"
- " $Y = 0 \rightarrow$ plant does not grow successfully"

Features:

- " X_1 : Average Rainfall"
- " X_2 : Temperature"

Our task is to learn a function that maps the two input features to these binary labels.

Now let's see what happens when we train a logistic regression model on this data.



- The red dots are still the true training labels.
- They remain fixed at 0 or 1.
- Model does NOT force its predictions to match them exactly — instead it learns a smooth probability surface that best fits the data

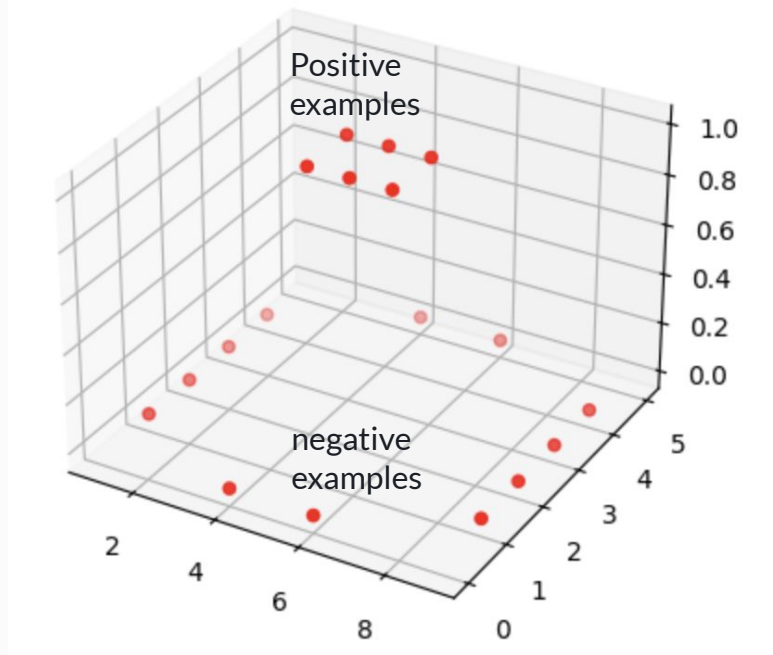
Non Linear Logistic Regression

Extending Logistic Regression

- As with Linear Regression, Logistic Regression can produce non-linear decision boundaries by introducing non-linear transformations of input features.
- This allows the model to handle more complex patterns in the data.

Example: Circular Pattern

- Positive examples are clustered in the center.
- Negative examples form a ring around the cluster.
- A simple linear decision boundary is insufficient.

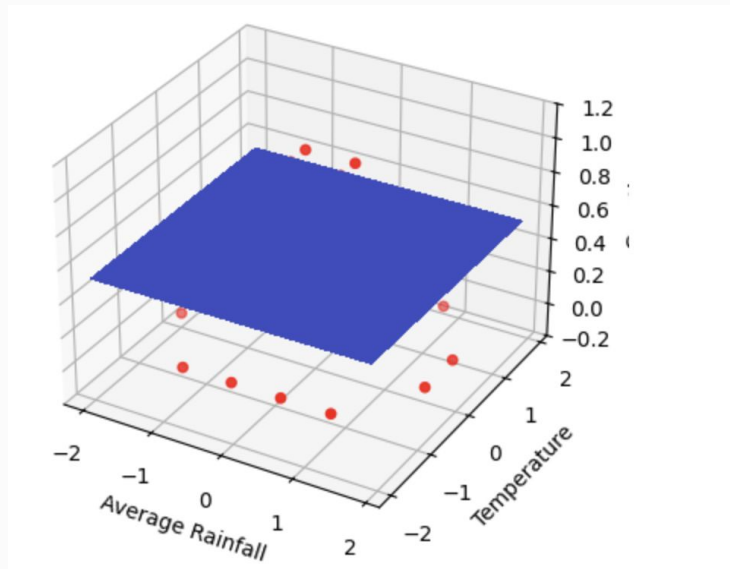


Non Linear Logistic Regression

Example: Circular Pattern

- Positive examples are clustered in the center.
- Negative examples form a ring around the cluster.
- A simple linear decision boundary is insufficient.

By introducing non-linear features (X^2, Y^2), a **2D non-linear boundary** can correctly separate the classes.



Solution: Non-linear Features

1. Problem Setup:

- Data: Positive examples clustered at the center, negatives forming a ring.
- Linear logistic regression **cannot separate** this pattern — produces ~50% probability across most points.

2. Solution: Feature Engineering

- Extend features to allow non-linear boundaries:

$$X_3 = X_1^2, \quad X_4 = X_2^2$$

- Now logistic regression can **learn a curved / circular decision boundary**.

3. Parameters (θ):

- $\theta_0 \rightarrow$ bias / intercept (shifts surface up/down)
- $\theta_1, \theta_2 \rightarrow$ linear effects of X_1 and X_2
- $\theta_3, \theta_4 \rightarrow$ non-linear effects of X_1^2 and X_2^2

Logistic Regression then operates on these transformed features to capture nonlinear separations.

Solution: Non-linear Features

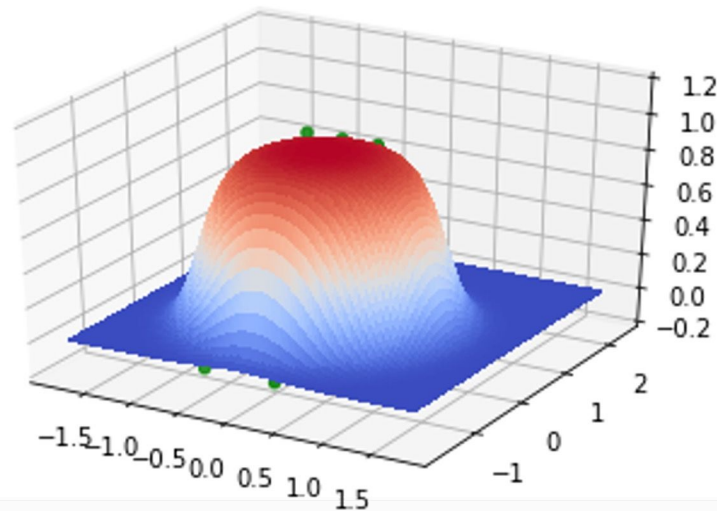
Extending Features:

$$X_3 = X_1^2, \quad X_4 = X_2^2$$

- Allows Logistic Regression to model a **non-linear, circular boundary**.

- Logistic Regression now becomes:

$$h_{\theta}(X) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_3 + \theta_4 X_4)}}$$



1. Axes:

- x-axis = feature X1
- y-axis = feature X2
- z-axis = predicted probability of Y = 1

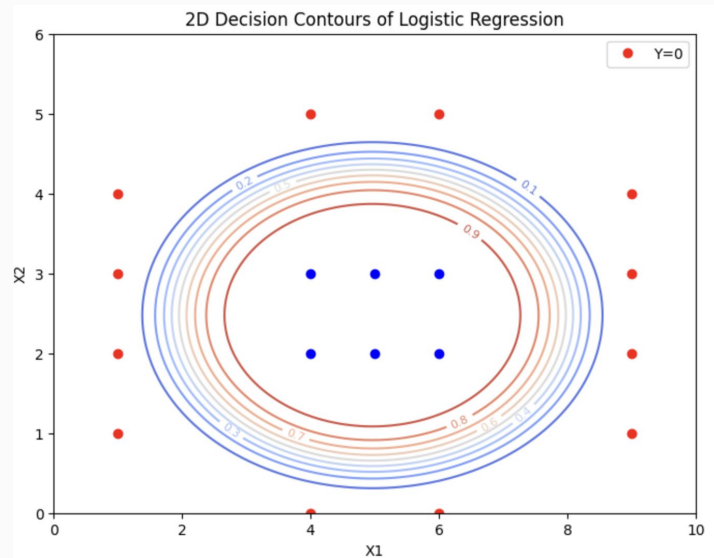
2. Surface color / height:

- Close to 1 → model predicts positive (green points above surface)
- Close to 0 → model predicts negative (green points near bottom)

3. Decision boundary:

- Anywhere the surface crosses $z = 0.5$ → 50% probability → threshold for classification
- Because we included X_1^2 and X_2^2 , the surface **curves to separate the positive cluster from the surrounding negatives**

Collapsed 2D graph of decision boundaries



1. Each contour line corresponds to a **predicted probability level** (0.0 $\rightarrow$ 1.0).
2. The **0.5 contour** is the **actual decision boundary** (threshold for classifying $Y=1$ vs $Y=0$).
3. Contours closer to 1 $\rightarrow$ model is more confident the point is positive.
4. Contours closer to 0 $\rightarrow$ model is confident the point is negative.

Non Linear Logistic Regression

Recap

- Basic logistic regression only models linear decision boundaries
- Our data has a circular pattern so linear model was not successful.
- Manual feature engineering = telling the model explicitly how to curve the boundary.
- Neural nets
 - letting the model figure out the right transformations from data automatically.
 - Neural networks generalize this idea and can automate non-linear feature discovery.
 - They can discover transformations that we manually defined here!

Multinomial Logistic Regression

- Multinomial Targets -

- From Binary to Multiclass Logistic Regression
- Up to now, our target variable Y has been binary (0 or 1).
- Many real-world problems have more than two classes:
 - Example: colors (red, green, blue)
 - Example: disease types (A, B, C, D)
- Standard Logistic Regression cannot directly handle more than two classes.
- Instead we train one model for each class and combine their predictions → OvR (One versus Rest) approach

Multinomial Logistic Regression

- One-Hot Encoding -

- One-Hot Encoding transforms a categorical variable with $k > 2$ classes into k binary variables.
- Each binary variable indicates presence (1) or absence (0) of one class.
- This allows Logistic Regression to treat each class independently while combining the outputs.
- Each column in one hot vector corresponds to target for one binary model

Example:

Original Y

One-Hot Encoding

Red

[1,0,0]

Green

[0,1,0]

Blue

[0,0,1]

- **Model Red** → target = first column (1 if Red, 0 otherwise)
- **Model Green** → target = second column (1 if Green, 0 otherwise)
- **Model Blue** → target = third column (1 if Blue, 0 otherwise)

Multinomial Logistic Regression

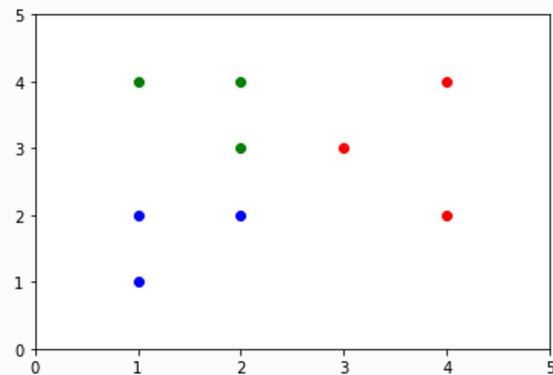
- Applying Logistic Regression to Multiclass Targets -

- Input features: X_1, X_2
- Target: Y with 3 classes
- Steps:
 1. One-Hot Encode $Y \rightarrow$ create 3 binary target columns
 2. Train a **separate logistic regression** for each class
 3. Combine predictions $\rightarrow$ **class with highest predicted probability is chosen**
- This method is called "**One-vs-Rest**" (OvR) classification

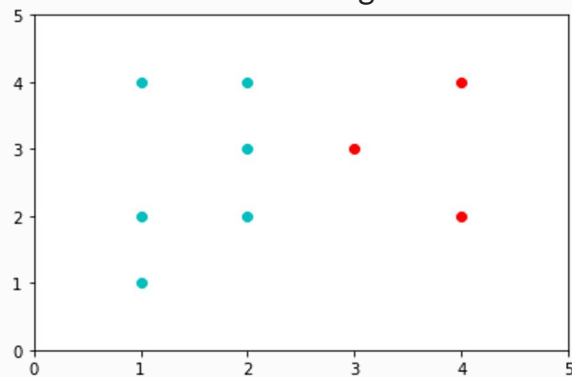
Sample	Model Red	Model Green	Model Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

- Each model is a **binary classifier**: "Is it this color or not?"
- At prediction time, each model outputs a probability, and we pick the **highest probability** as the predicted class.

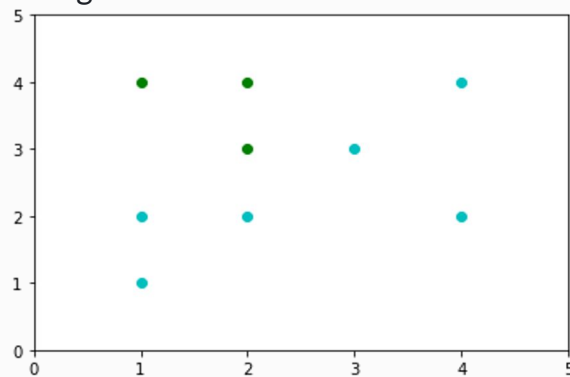
Multinomial Logistic Regression



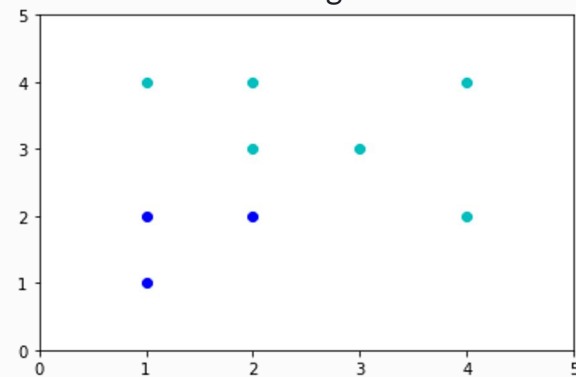
Red vs blue & green



green vs red & blue



blue vs red & green

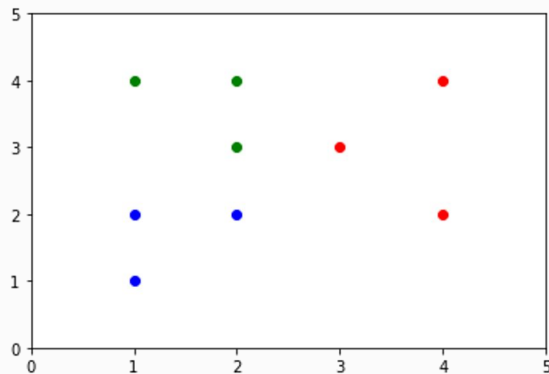


Multinomial Logistic Regression

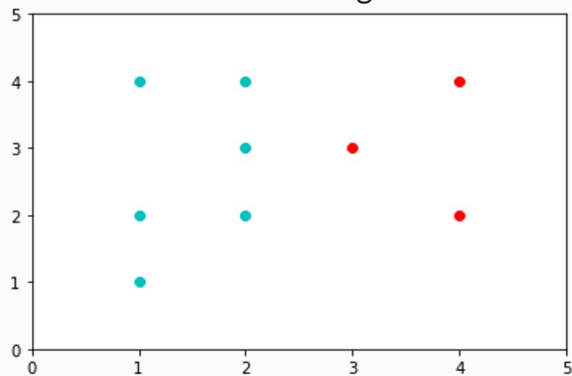
Example probabilities

- Red = 0.5589
- Green = 0.3561
- Blue = 0.2000

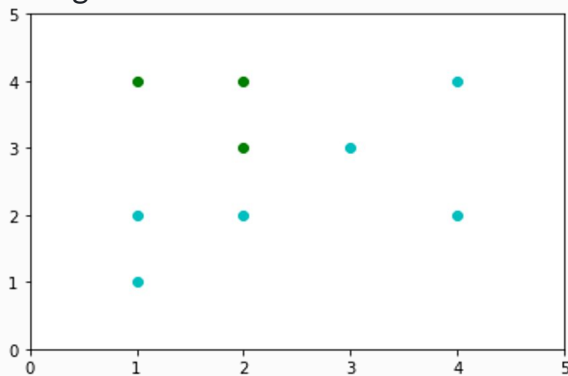
Predicted class would be red!



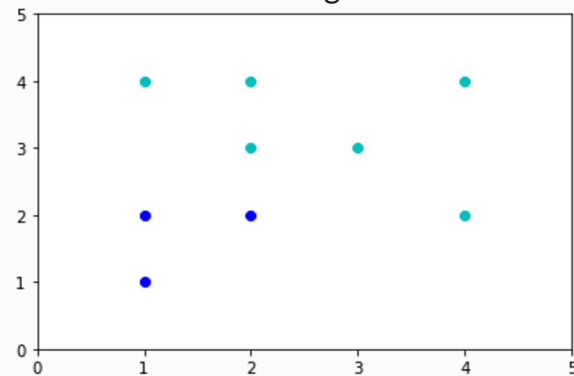
Red vs blue & green



green vs red & blue



blue vs red & green

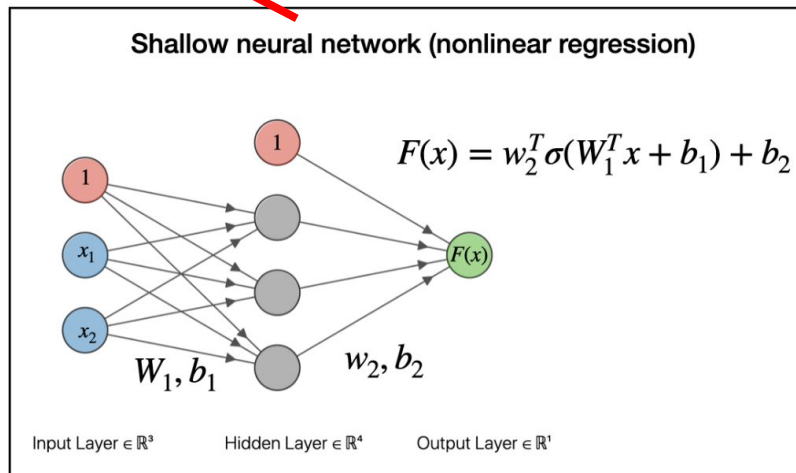
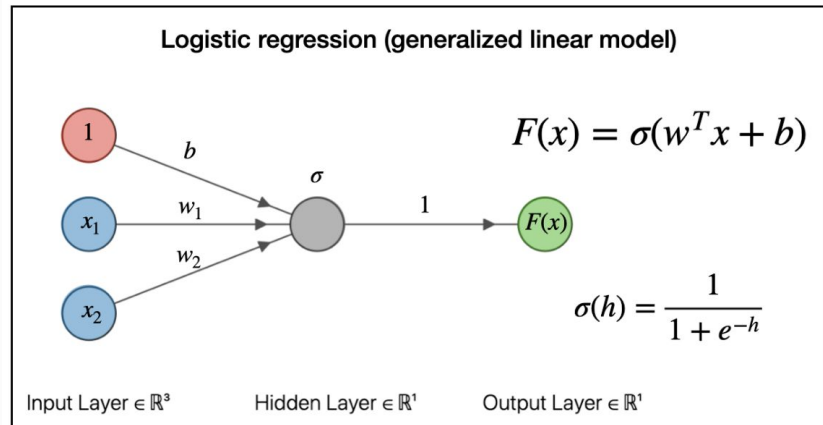


Multinomial Logistic Regression

We will take a different approach in neural networks where instead of training 3 independent models, we train one model with one set of weights but with outputs for all classes.

Next Week

Bonus concepts on overfitting and underfitting



Project Proposal

- Expected to submit a project proposal by Week-5 (25th of Feb, in~2 weeks)

Lab Work

- Download the lecture notebook for Week 3 and run it in Google Colab or on your own Jupyter server.
- Design your own classification task with a clear binary distinction based on two input variables. You will need to use the multivariate version of the code to work through this model.
- Try to construct a dataset that exhibits a clear underlying pattern, but still presents the training algorithm with a greater challenge when learning a clean classification boundary.

APPENDIX

Derivative of Logistic Cost Function

[Back](#)

1. Start with the logistic cost function

The cost for a single training example is:

$$\text{cost}_{\text{logistic}}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

The overall cost is the **average over all examples**:

$$J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m \text{cost}_{\text{logistic}}(h_{\theta}(x^{(i)}), y^{(i)})$$

2. Take the partial derivative with respect to θ_j

$$\frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_j} \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

Derivative of Logistic Cost Function

3. Apply the chain rule

- The derivative of $-y \log(h_\theta)$ with respect to θ_j :

$$\frac{\partial}{\partial \theta_j} [-y \log(h_\theta)] = -y \cdot \frac{1}{h_\theta} \cdot \frac{\partial h_\theta}{\partial \theta_j}$$

- The derivative of $-(1 - y) \log(1 - h_\theta)$:

$$\frac{\partial}{\partial \theta_j} [-(1 - y) \log(1 - h_\theta)] = -(1 - y) \cdot \frac{-1}{1 - h_\theta} \cdot \frac{\partial h_\theta}{\partial \theta_j} = \frac{1 - y}{1 - h_\theta} \cdot \frac{\partial h_\theta}{\partial \theta_j}$$

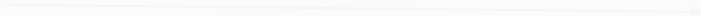
4. Combine the two terms

$$\frac{\partial}{\partial \theta_j} \text{cost}_{\text{logistic}} = \left(-\frac{y}{h_\theta} + \frac{1 - y}{1 - h_\theta} \right) \frac{\partial h_\theta}{\partial \theta_j} = \frac{h_\theta - y}{h_\theta(1 - h_\theta)} \cdot h_\theta(1 - h_\theta)x_j = (h_\theta - y)x_j$$

- Key step: $\frac{\partial h_\theta}{\partial \theta_j} = h_\theta(1 - h_\theta)x_j$
- The $h_\theta(1 - h_\theta)$ cancels with the denominator, leaving $(h_\theta - y)x_j$

Derivative of Logistic Cost Function

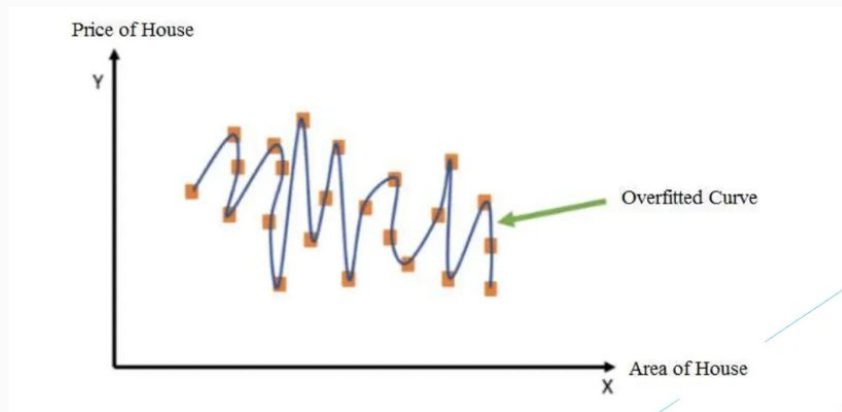
5. Sum over all examples

$$\frac{\partial}{\partial \theta_j} J_{\text{logistic}}(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$


Bonus Concepts

Understanding Overfitting

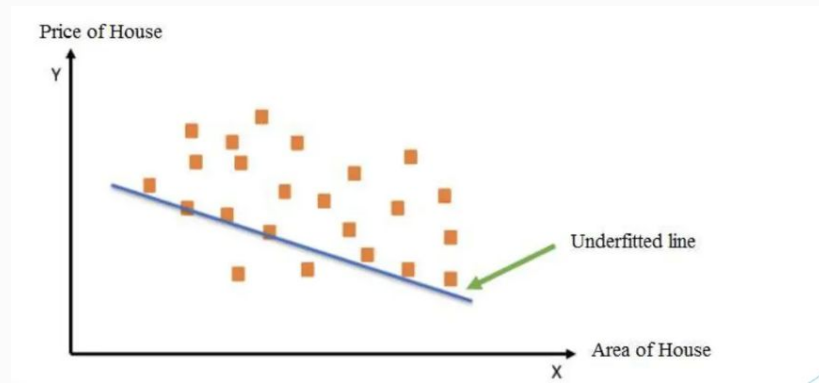
- Overfitting ~ Model Too Complex
- Definition: The model fits the training data too closely, including noise.
- Symptoms:
 - High accuracy on training data
 - Low accuracy on test/validation data
- Causes:
 - Too complex model (e.g., very deep network for small dataset)
 - Too many features, not enough data



Bonus Concepts

Understanding Underfitting

- Underfitting ~ Model Too Simple
- Definition: Model cannot capture the underlying patterns in the data.
- Symptoms:
 - Low accuracy on training data
 - Low accuracy on test data
- Causes:
 - Model too simple (e.g., Linear Regression for complex non-linear data)
 - Too few features or insufficient training



Bonus Concepts

Importance of Balancing

- Goal: Achieve good performance on both training and unseen data
- Techniques to avoid over/underfitting
 - Cross-validation
 - Regularization (L1, L2)
 - Pruning (for trees)
 - Early stopping (for neural networks)
 - Collect more data

